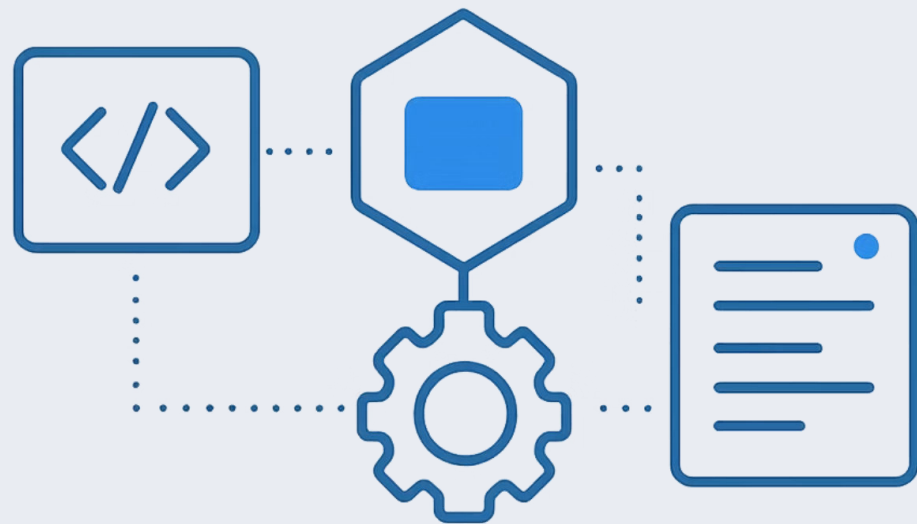




Code Optimization

Compilers that apply code-improving transformations are called **optimizing compilers**. Optimizations make programs run faster, take less space, or both – classified into [machine independent](#) and [machine dependent](#) categories.

Types of Optimization

Machine Independent

Improves target code without considering properties of the target machine. Eliminates redundancies and reduces code size universally.

Machine Dependent

Based on register allocation and utilization of special machine-instruction sequences specific to the target hardware.

Criteria for Code Improvement

1

Maximum Benefit, Least Effort

Best transformations yield the most benefit for the least effort.

2

Preserve Meaning

Optimization must never change program output or introduce new errors.

3

Measurable Speedup

Must speed up programs by a measurable amount on average.

4

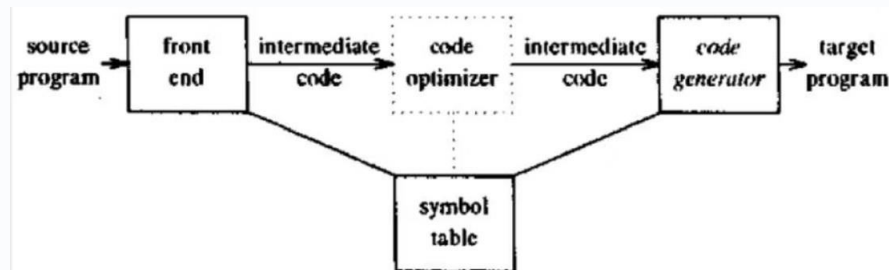
Worth the Effort

"Peephole" transformations must repay the compiler writer's investment.

Position of the Code Optimizer

Compiler Pipeline

The code optimizer sits between two passes of intermediate code generation. It uses the **symbol table** alongside the front end and code generator to produce optimized intermediate code before final target program generation.



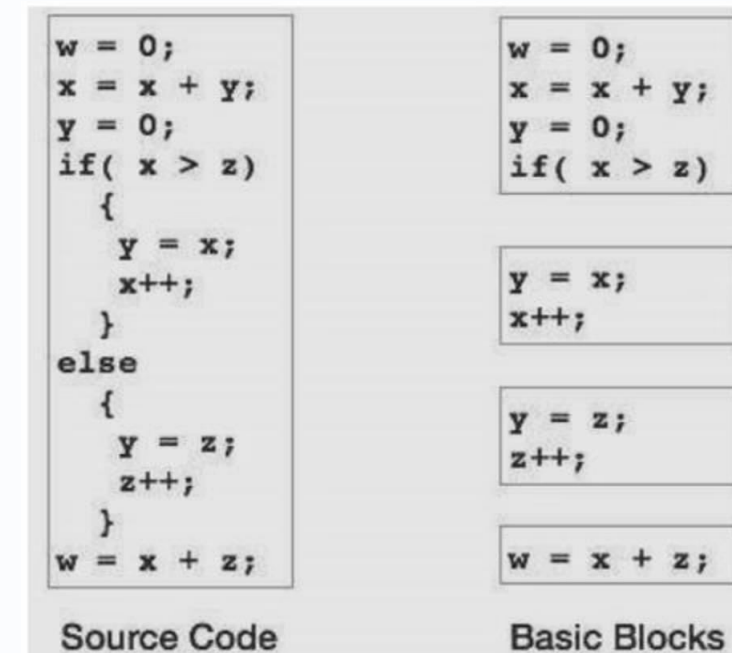
- ① Flow analysis – both control flow and data flow – is a fundamental prerequisite for code improvement.

Basic Blocks

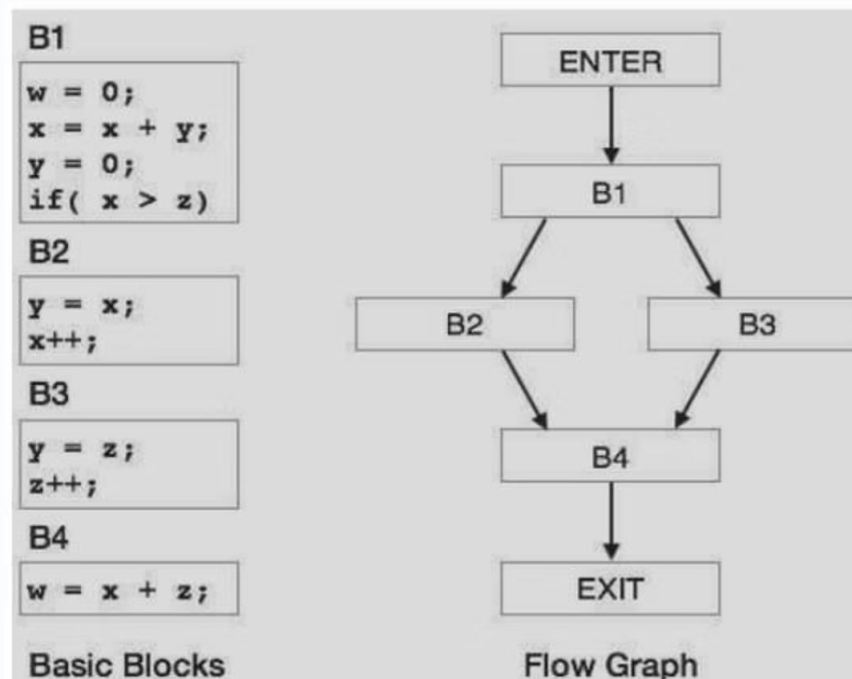
A **basic block** is a sequence of instructions always executed in order, with no internal jumps. Programs contain basic blocks in constructs like IF-THEN-ELSE, SWITCH-CASE, and loops (DO-WHILE, FOR, REPEAT-UNTIL).

Identification Algorithm

- First statement of a program
- Targets of any branch (conditional/unconditional)
- Statements following any branch statement



Control Flow Graph



A **control flow graph (CFG)** depicts how program control passes among basic blocks. It is a key tool for optimization — helping locate unwanted loops and unreachable code.

- CFG construction is part of Control Flow Analysis (CFA), which precedes Data Flow Analysis (DFA).

Machine Independent Optimization

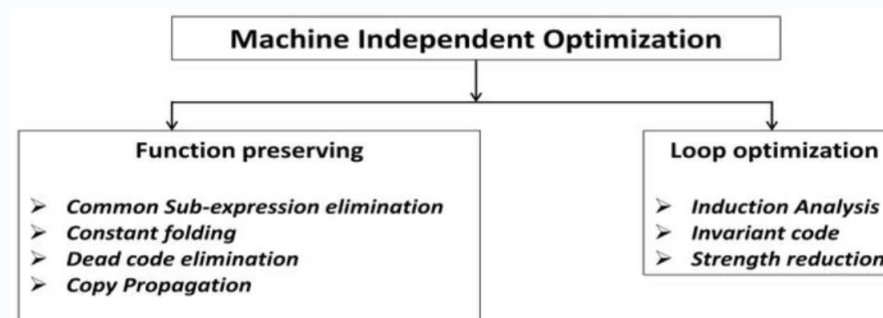
Optimizes intermediate code by eliminating redundancies, reducing lines of code, and removing useless or repeated code — applicable on **any processor** regardless of machine specifications.

Function Preserving

- Common Subexpression Elimination
- Constant Folding
- Dead Code Elimination
- Copy Propagation

Loop Optimization

- Induction Analysis
- Invariant Code
- Strength Reduction



Common Subexpression Elimination & Constant Folding

Common Subexpression Elimination

If an expression is computed multiple times without change, eliminate the duplicate. E.g., `t1=x*z` and `t4=x*z` — `t4` is redundant and can be removed.

Constant Folding

Replace compile-time expressions with their values. E.g., `5+7+c` → `12+c`, saving runtime evaluation.

Constant Propagation

If a variable holds a constant, substitute it directly. E.g., `a=5;`
`c=a*2` → `c=5*2` → `c=10`.

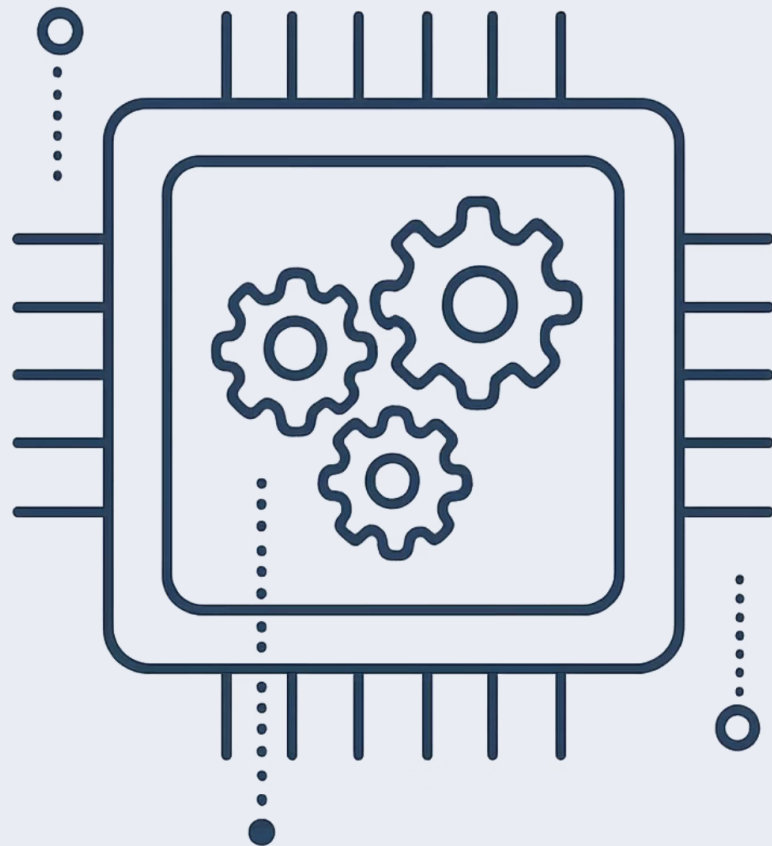
Dead Code Elimination & Copy Propagation

Dead Code Elimination

Remove code that is never executed or whose result is never used. E.g., `int x=a+23;` where `x` is never referenced — safely deleted without affecting the program.

Copy Propagation

Where assignments like `x=y` exist, use `y` directly in place of `x`. Eliminates unnecessary copy statements and reduces variable usage.



II. Loop Optimization

A brief introduction to one of the most important areas for compiler optimizations — **loops**, especially inner loops where programs spend the bulk of their execution time. The running time of a program may be improved if we decrease the number of instructions in an inner loop, even if we increase the amount of code outside that loop.

1. Code Motion

Moves loop-invariant code outside the loop to reduce repeated computation.

2. Induction-Variable Elimination

Replaces redundant induction variables from inner loops with a single canonical variable.

3. Reduction in Strength

Replaces an expensive operation by a cheaper one, such as a multiplication by an addition.

1) Code Motion

An important modification that decreases the amount of code in a loop is **code motion**. This transformation takes an expression that yields the same result independent of the number of times a loop is executed — a **loop-invariant computation** — and places the expression *before* the loop. Note that the notion "before the loop" assumes the existence of an entry for the loop.

Before Code Motion

```
while (i <= limit-2)
    /* statement does not
       change limit */
```

Here, `limit-2` is recomputed on every iteration, even though `limit` never changes inside the loop.

After Code Motion

```
t = limit-2;
while (i <= t)
    /* statement does not
       change limit or t */
```

The loop-invariant computation `limit-2` is hoisted out of the loop and stored in a temporary variable `t`, evaluated only once.

2) Induction Variables

A variable is called an **induction variable** if its value is altered within the loop by a loop-invariant value. Induction variable elimination removes redundant induction variables, simplifying the loop body and reducing overhead.

Before Elimination

```
int a[SIZE];
int b[SIZE];
void f(void) {
    int i1, i2, i3;
    for (i1=0, i2=0, i3=0;
        i1 < SIZE; i1++)
        a[i2++] = b[i3++];
    return;
}
```

Three induction variables `i1`, `i2`, and `i3` all increment in lockstep — `i2` and `i3` are redundant.

After Elimination

```
int a[SIZE];
int b[SIZE];
void f(void) {
    int i1;
    for (i1=0;
        i1 < SIZE; i1++)
        a[i1] = b[i1];
    return;
}
```

After elimination, only the canonical induction variable `i1` remains, directly indexing both arrays. The loop is cleaner and faster.

3) Reduction in Strength

Reduction in strength replaces expensive operations by equivalent cheaper ones on the target machine. Certain machine instructions are considerably cheaper than others and can often be used as special cases of more expensive operators.

Exponentiation → Multiplication

x^2 is invariably cheaper to implement as $x*x$ than as a call to an exponentiation routine.

Multiply/Divide by Power of 2 → Shift

Fixed-point multiplication or division by a power of two is cheaper to implement as a **bit shift** operation.

Floating-Point Division → Multiplication

Floating-point division by a constant can be implemented as multiplication by a constant, which may be cheaper on the target machine.

Original Loop

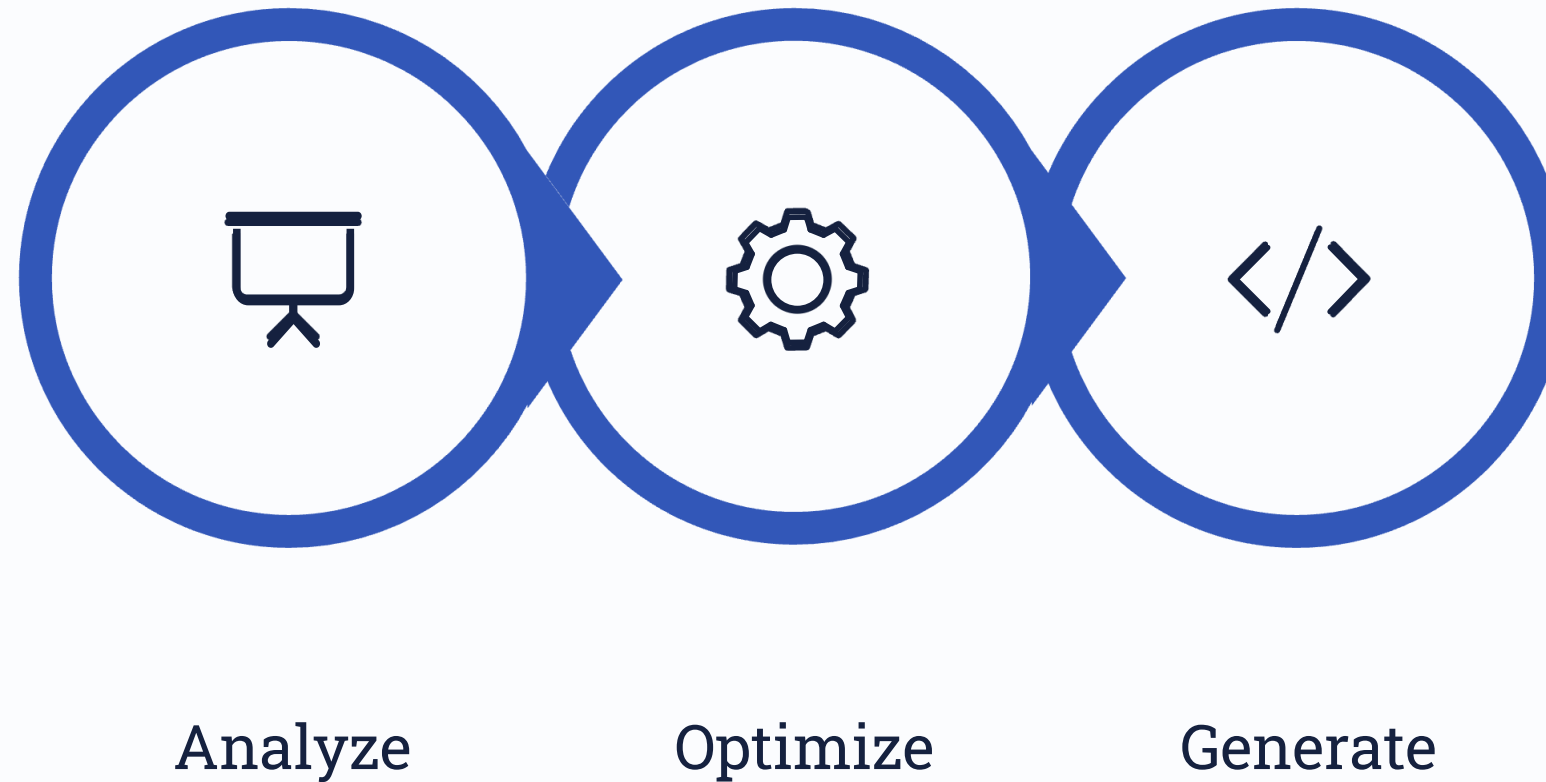
```
while (i < 10) {  
    j = 3*i + 1;  
    a[j] = a[j] - 2;  
    i = i + 2;  
}
```

After Strength Reduction

```
s = 3*i + 1;  
while (i < 10) {  
    j = s;  
    a[j] = a[j] - 2;  
    i = i + 2;  
    s = s + 6;  
}
```

The variable `s` replaces the multiplication `3*i+1` inside the loop. Since `i` increments by 2, `s` increments by 6 — replacing a multiply with an add.

Key Takeaways



Code optimization is a critical compiler phase — preserving program semantics while maximizing speed and minimizing size through machine independent and machine dependent transformations.